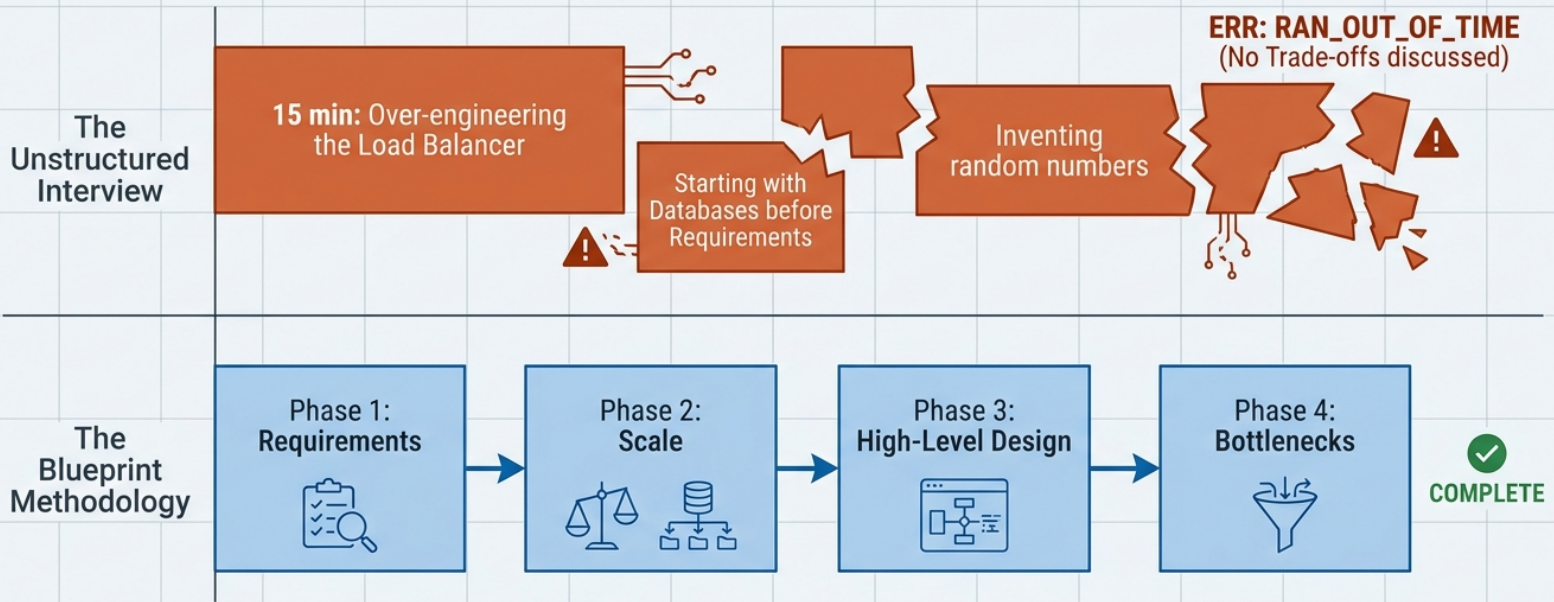


# THE 45-MINUTE ARCHITECTURE BLUEPRINT A REPEATABLE FRAMEWORK FOR SYSTEM DESIGN INTERVIEWS

---

PROTOCOL: SYSTEM\_DESIGN  
VERSION: 1.0.0  
STATUS: READY FOR DEPLOYMENT

# The 45-Minute Trap



The interviewer evaluates your ability to structure ambiguity under strict time constraints. You need a repeatable process, not memorized architectures.

# The 4-Phase System Design Protocol

## Phase 1:

### Requirements (5 min)

Define functional/non-functional specs.  
Clarify scope.  
Identify constraints.



## Phase 2:

### Scale Estimation (5 min)

Back-of-envelope calculations.  
QPS, Storage, Bandwidth.  
Identify scale challenges.



## Phase 4: Bottlenecks & Deep Dives

### Deep Dives (20 min)

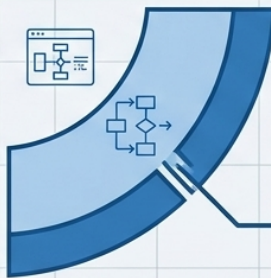
Analyze single points of failure. Optimize critical paths. Discuss trade-offs.  
Deep dive into one component.



## Phase 3:

### High-Level Design (15 min)

Diagram core components.  
Define data flow.  
Select technologies.  
Sketch architecture.



## Case Study:

We will prove this methodology by designing a global mapping system (Google Maps) from scratch, strictly following the 45-minute constraint.



# Phase 1: Triage the Ambiguity

## Requirements Triage Matrix

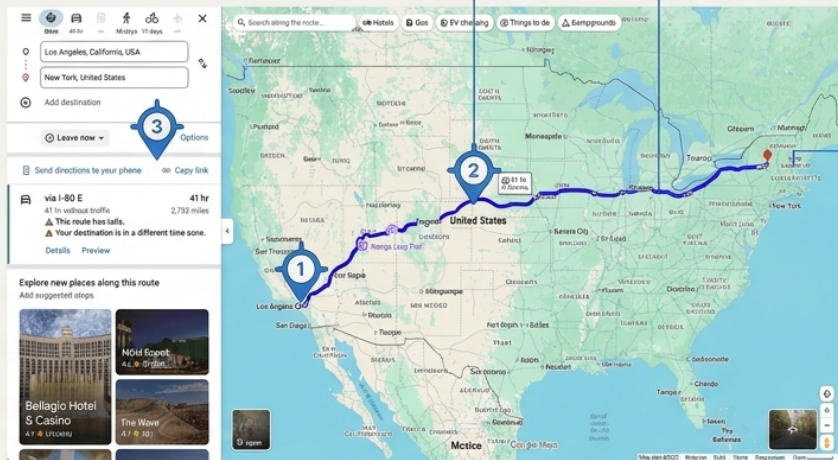
| Functional (What it does)                                                                                                                                                                                                                                                                                     |  | Non-Functional (How well it performs) |       |                         |         |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|---------------------------------------|-------|-------------------------|---------|
|  — User journeys<br> — Content creation<br> — Interactions |  | Latency                               | ----- | Availability            | ___/___ |
|                                                                                                                                                                                                                                                                                                               |  | Consistency                           | ----- | Geographic Distribution | ___/___ |
|                                                                                                                                                                                                                                                                                                               |  |                                       |       |                         |         |
| LIMIT: Cap scope at 3-4 core features.                                                                                                                                                                                                                                                                        |  |                                       |       |                         |         |

## Rules of Engagement

 **AVOID UNNECESSARY QUESTIONS.** Ask only questions that materially change the architecture. Define the read/write ratio, user volume, and geographic spread.



# Case Study: Mapping the Requirements



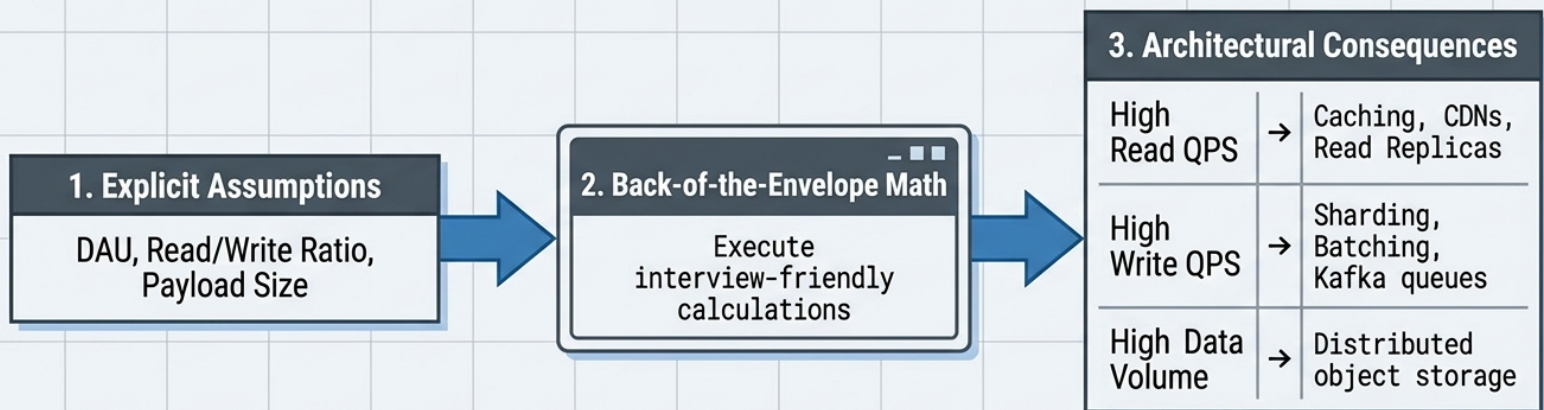
## Functional Scope (The Targets)

1. **Location Identification:** Pinpoint latitude/longitude.
2. **Route Recommendation:** Optimal path by distance/time.
3. **Navigation Directions:** Step-by-step guidance.

## Non-Functional Scope (The Guardrails)

- Extreme Availability & Scalability (enterprise integrations like Uber).
- Low Latency: ETA calculation strictly within 2–3 seconds.
- High Geographic Distribution: Dictates regional data residency and CDN usage.

## Phase 2: The Consequence Pipeline



**Core Principle: Never invent numbers. Derive them.  
Every calculation must output an architectural decision.**

# Case Study: Calculating the Global Grid

## // ASSUMPTIONS

Daily Active Users: 32,000,000

Requests/user/day: 50

Server Capacity: 64,000 RPS / server

## // CALCULATING TOTAL RPS

$(32M * 50) / 86400 \approx 18,518$  RPS

## // CALCULATING SERVER LOAD

32,000,000 RPS peak / 64,000 = 500 Servers Required

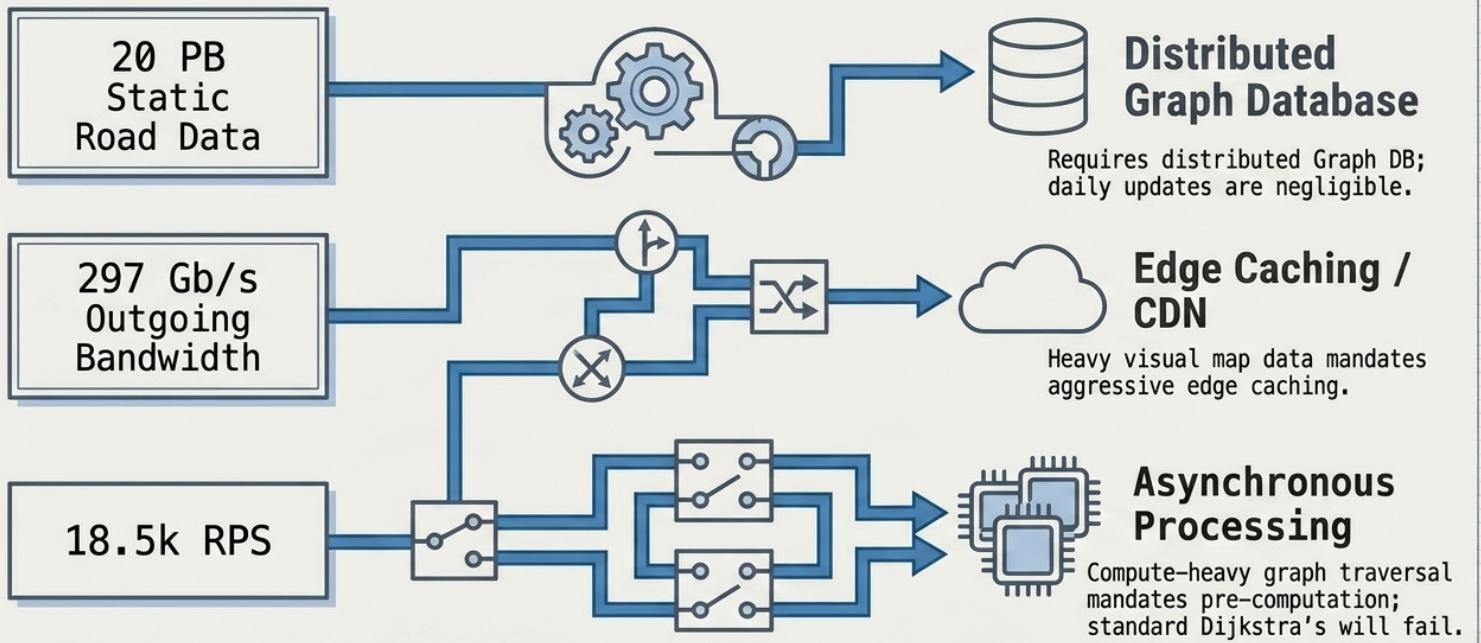
## // CALCULATING BANDWIDTH (200B query / 2MB response)

Incoming:  $18,518 * 200B \approx 29.6$  Mb/s

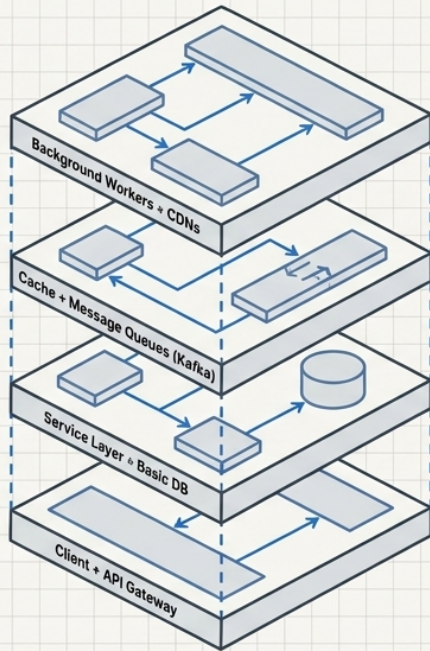
Outgoing:  $18,518 * 2MB \approx 297$  Gb/s



# Locking in the Architecture



# Phase 3: The Progressive Disclosure Stack



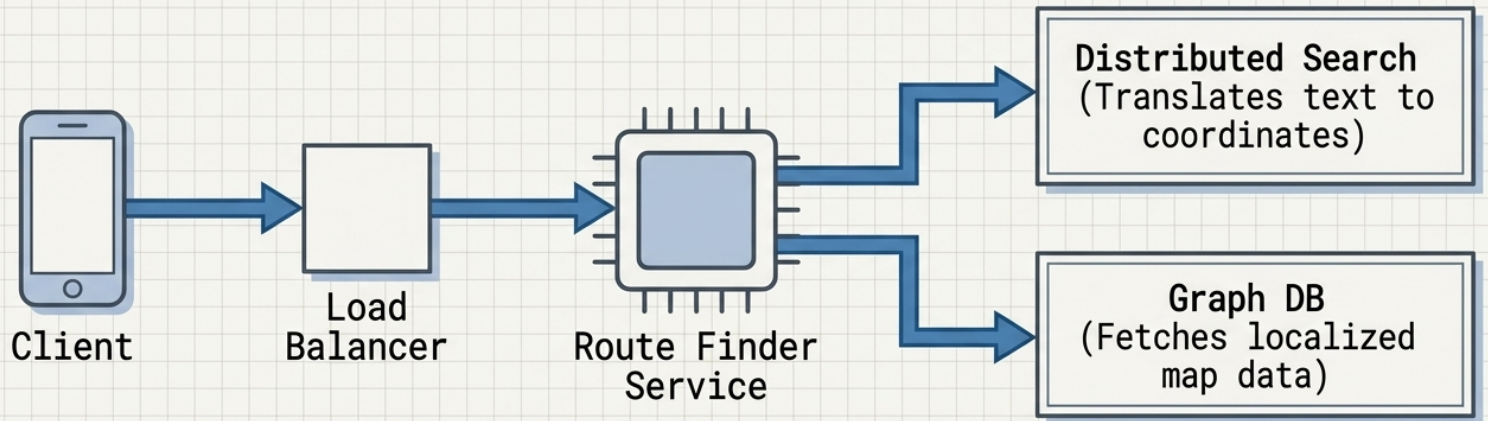
## The Component Checklist

- ☐ What problem does this solve?
- ☐ Why is it needed at our exact scale?
- ☐ What happens if it fails?
- ☐ Do NOT blindly include every component.

## Core Principle:

Start with a single-user baseline. Only introduce distributed-system complexity to solve a specific scale problem.

# Baseline Architecture: The Single Journey

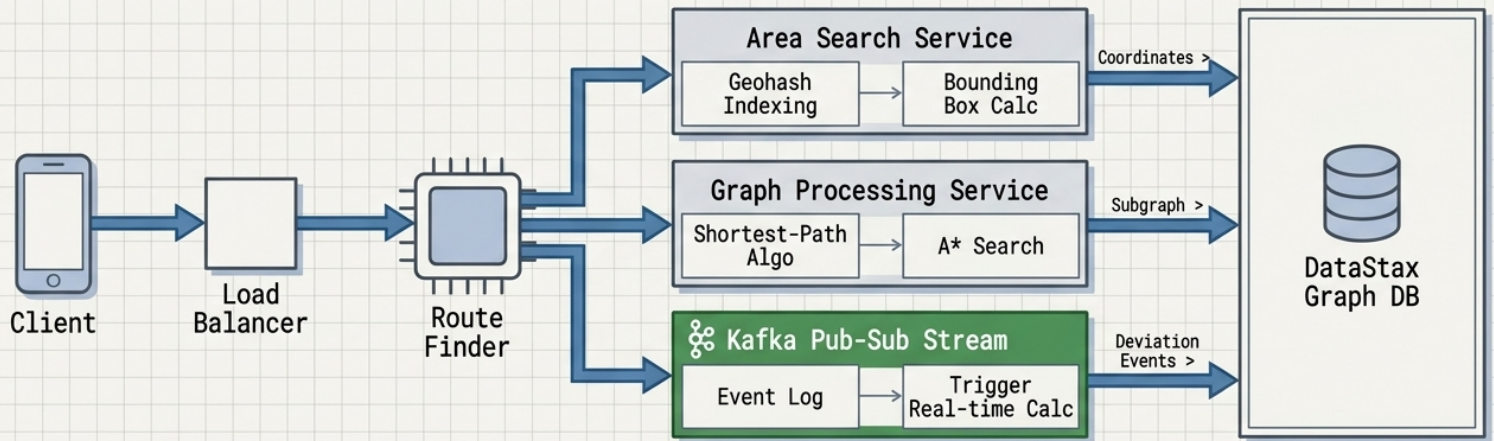


## Constraint Check:

Constraint Check: This works perfectly for one user in one city. It breaks instantly at 32M DAU across multiple regions.



# Scaling to the Global Grid



## Area Search Service

Requests specific coordinates to create a bounding box, isolating the search space.

## Graph Processing Service

Runs the shortest-path algorithm exclusively on the isolated subgraph.

## Pub-Sub (Kafka)

Captures real-time route deviation events to trigger immediate recalculation without blocking the main thread.

# Architectural Defense Matrix

| [Component]         | [Problem Solved]                                                                   | [Failure Impact]                       | [Alternative]                                |
|---------------------|------------------------------------------------------------------------------------|----------------------------------------|----------------------------------------------|
| Kafka (Pub-Sub)     | Decouples live navigation events from synchronous route calculation                | Delayed ETA updates, no broken routing | RabbitMQ, SQS                                |
| Graph DB (DataStax) | Natively stores intersections (vertices) and roads (edges) for efficient traversal | Core routing failure                   | Relational DB with recursive CTEs (Too slow) |
| Key-Value Store     | O(1) lookups mapping segment IDs to specific host servers                          | Segment routing failure                | Redis, Memcached                             |

# The Core API Contracts

|                                                                                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------------------|
| Endpoint 1: Route Optimization                                                                                                             |
| GET /v1/routes                                                                                                                             |
| Payload: {source: string, destination: string, transport_type: enum}<br>Response: {route_id: uuid, eta_seconds: int, distance_meters: int} |
| Endpoint 2: Real-time Navigation                                                                                                           |
| POST /v1/navigation/update                                                                                                                 |
| Payload: {current_lat: float, current_long: float, route_id: uuid}<br>Response: {status: 'on_track   deviated', next_instruction: string}  |

Design Note: APIs are kept strictly focused on the core functional requirements established in Phase 1.



# Phase 4: The Critical Bottleneck - Global Graph Traversal



## The Reality

The global road network contains billions of vertices. Running Dijkstra's or A\* algorithms on the entire dataset at 18,518 requests per second is computationally impossible.

## The Interview Pivot

This is where senior candidates separate from mid-level engineers. We must restructure the data layer to avoid traversing the full graph during a live query.

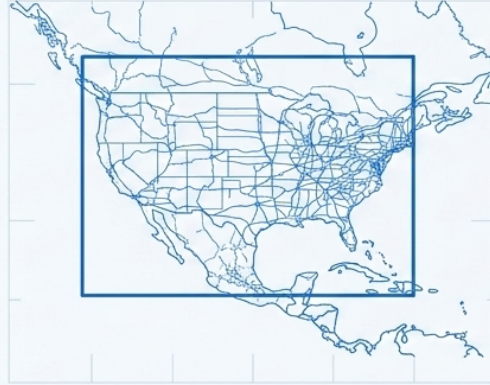
# Spatial Partitioning (Segments)

## Macro-to-Micro

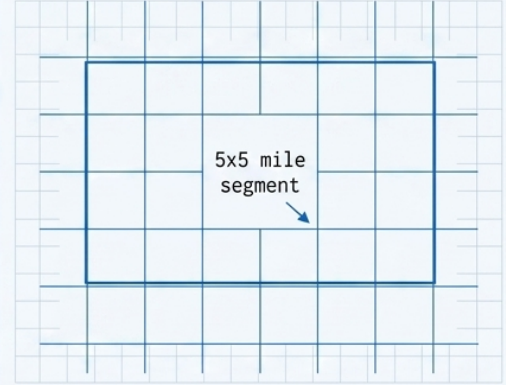
Panel 1: Global View



Panel 2: Region Zoom



Panel 3: Segment Grid

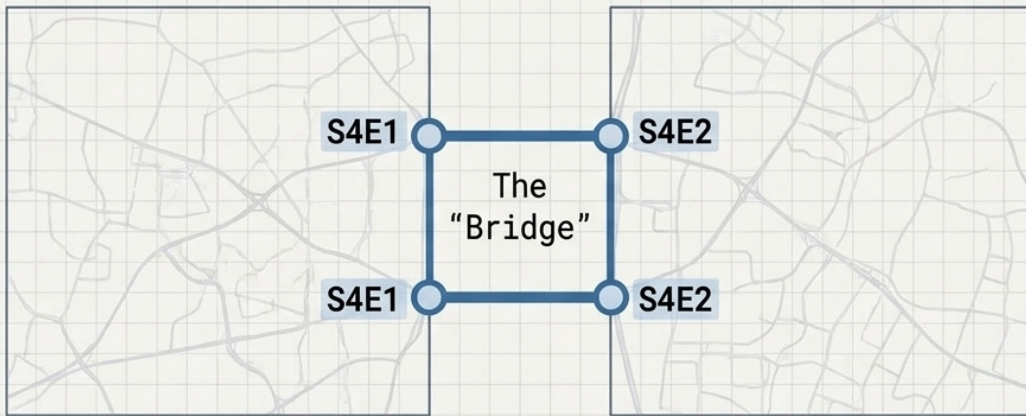


## The Mechanism

- Partition the global map into manageable subgraphs (segments).
- Each segment can be loaded entirely into RAM.
- Shortest paths *within* a segment are pre-computed offline and cached (distance, time, sequence of vertices).
- Key-Value lookup maps SegmentID -> ServerID. Live queries never block for internal segment routing.

# The Exit Point Bridge

---



## Cross-Segment Routing

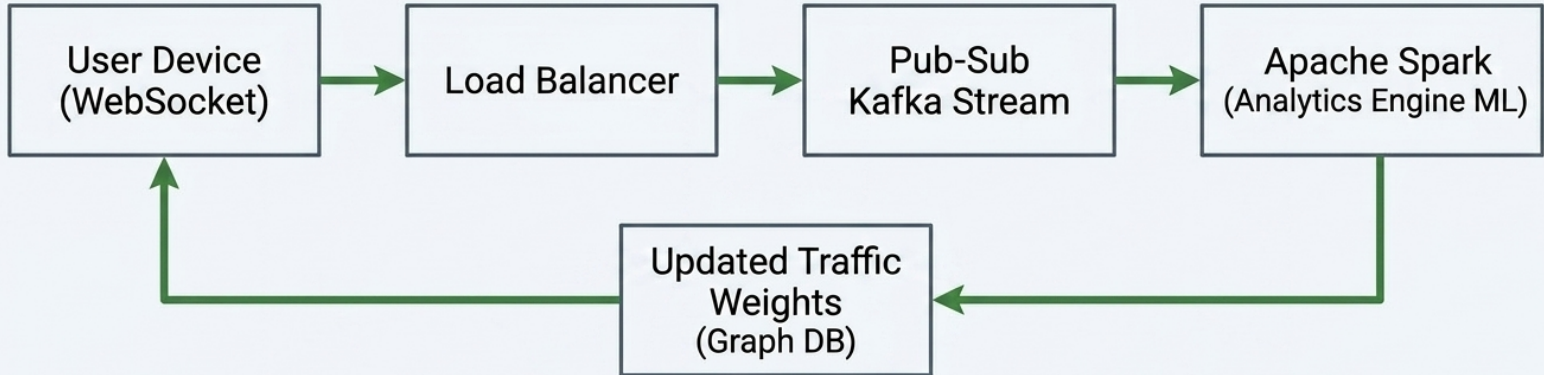
---

1. Filter the global map using the Haversine formula (aerial distance) to create a limited bounding box around the source and destination.
2. Identify the exit points (shared boundary vertices) of the involved segments.
3. Build a high-level graph using *only* the exit points and their pre-calculated paths.
4. Run the shortest path algorithm on this massively simplified overlay graph.



# Real-Time ETA Calibration

---



## The Distributed Pipeline

---

- Persistent two-way WebSocket connections stream live GPS data to the servers.
- Kafka aggregates the firehose of location streams (device, time, coordinates).
- Apache Spark processes the stream, detecting traffic density, construction, and road closures.
- The analytics engine recalculates edge weights and publishes ETA adjustments back to active navigators in real-time.

# The Master Blueprint Synthesized

| Phase               | Goal       | Key Deliverable          | Required Artifact                        | Fatal Pitfall                   |
|---------------------|------------|--------------------------|------------------------------------------|---------------------------------|
| Phase 1<br>(0-5m)   | Triage     | Core Scope               | Functional /<br>Non-Functional<br>Matrix | Asking too many<br>questions    |
| Phase 2<br>(5-10m)  | Boundaries | Target<br>Architecture   | Pipeline Math                            | Numbers without<br>context      |
| Phase 3<br>(10-25m) | Structure  | Component<br>Layout      | Stack + APIs                             | Premature<br>optimization       |
| Phase 4<br>(25-45m) | Mastery    | Bottleneck<br>Resolution | Deep Dive<br>Visual                      | Staying at the<br>surface level |